

Un agente investigador sobre informes 10-K

MIAX · Instituto BME · Bloque V — LLMs aplicados a Finanzas Alonso, Pietro y Raúl

Todas las tablas de este documento salen de `resultados/` y se regeneran con `agente-10k informe`. Todas las tablas salen de `resultados/`, que a su vez sale de ejecutar el repositorio. No hay ninguna cifra escrita a mano: si una tabla falta, aquí aparece qué orden la genera.

1. Qué hace el sistema

Un agente ReAct responde preguntas sobre los 10-K de seis tecnológicas (NVDA, MSFT, AAPL, GOOGL, META, AMZN) en FY2024 y FY2025 — 48 secciones, unos 650.000 tokens — y cita de dónde sale cada dato. Tiene cuatro herramientas con coste y precisión muy distintos:

herramienta	qué da	coste
<code>list_available</code>	qué emisores y ejercicios hay	gratis
<code>get_xbrl_fact</code>	el hecho XBRL exacto	gratis y exacto
<code>search_filings</code>	los fragmentos más parecidos a una consulta	barato, difuso
<code>read_section</code>	una sección entera, hasta cuarenta páginas	caro

La tarea central no es contestar: es **elegir bien**. Una cifra de los estados financieros se pide a XBRL, donde es exacta y gratuita; un matiz de los factores de riesgo hay que buscarlo en el texto. Un agente que lee la sección 8 entera para dar un beneficio por acción ha acertado por el camino equivocado, y en nuestra tabla eso cuenta como fallo.

La respuesta sale siempre como `RespuestaFinanciera`: prosa, cifra, unidad, emisor, ejercicio, fuente, cita literal y `chunk_id`. Añadimos dos campos — `concept_xbrl` y `motivo_sin_dato` — que el contrato permite porque llevan valor por defecto: el primero deja que el guardarraíl contraste contra el hecho exacto en lugar de adivinar qué concepto quiso decir el modelo, y el segundo distingue «no está en el corpus» de «no lo he sabido encontrar», que no es lo mismo.

2. El golden set

Veinte preguntas en `golden/golden_set.jsonl`, repartidas entre extractivas, numéricas y comparativas, con dos huecos deliberados: preguntas cuyo dato **no está** en el corpus y cuya respuesta correcta es decirlo.

La verdad de las extractivas se ancla a una **frase literal** del informe, nunca a un `chunk_id`. El identificador cambia en cuanto se re-trocea el corpus, y quien mejore el troceado no puede salir penalizado por haberlo mejorado. El `recall@k` se mide comprobando que esa frase aparece en alguno de los `k` fragmentos recuperados **y que el fragmento es del emisor y el ejercicio que la pregunta pide**: sin esa segunda comprobación, una frase que se repite en dos informes contaría como acierto en el equivocado.

`golden/validador.py` comprueba el esquema, las familias, los huecos y las escalas antes de que ninguna pregunta entre en una medición.

3. Cómo se evalúa

Tres evaluadores independientes miran cada respuesta:

- **cita** — el texto citado tiene que existir literalmente en el fragmento que dice; se compara con espaciado y comillas normalizados, no carácter a carácter;
- **cifra** — se acepta con una tolerancia única, 0,5 % o 1 USD, la que sea mayor, y se comprueba la unidad y la escala: confundir millones con miles es el fallo más frecuente y el más silencioso;
- **trayectoria** — qué herramientas usó y en qué orden.

El **acierto** exige las dos cosas: respuesta correcta **y** camino correcto. La tabla lleva además la columna de *acierto por el camino equivocado*, que es la que enseña cuánto de la nota es suerte.

Las preguntas hueco tienen su propia columna y su propio fallo: **alucinación sobre hueco**, dar una cifra donde la respuesta correcta era que el dato no está. Y su simétrico, la **abstención indebida**: decir «no hay dato» cuando sí lo había. Un sistema que no contesta nunca no alucina y no vale para nada; hacen falta las dos columnas para que ninguna de las dos trampas pase desapercibida.

Una limitación que conocemos

La tolerancia absoluta de 1 USD está pensada para importes de miles de millones, donde un dólar es ruido. Para una magnitud por acción es demasiado laxa: con un beneficio por acción de 11,86 USD, decir 12 entra en tolerancia, y es un error del 1,2 %. Lo descubrimos ensayando sobre un clon limpio — `get_xbrl_fact` redondeaba el BPA a entero y el modelo copiaba el 12—. Arreglamos la herramienta, pero **no** tocamos la tolerancia: cambiar la vara de medir después de congelar el baseline haría que las dos columnas de la tabla no se midieran igual. Lo correcto a futuro es una tolerancia por unidad.

Veinte preguntas no son una muestra grande

Con veinte preguntas, subir de 12 a 14 aciertos es mover dos preguntas. Por eso cada proporción va con su **intervalo de Wilson** —no la normal, que con n pequeño se sale de $[0, 1]$ y da anchura cero cuando se acierta todo— y cada comparación entre dos sistemas se contrasta con **McNemar exacto**: pareado, porque los dos sistemas responden las mismas preguntas, y exacto, porque la aproximación de chi-cuadrado pide unos veinticinco pares discordantes y aquí hay cuatro o cinco.

4. El retrieval, paso a paso

Cada fila es una configuración; el runner es el mismo. Añadir una fila a esta tabla es añadir una tupla en `retrieval/fabrica.py`, nunca escribir código.

configuración	recall@1	recall@3	recall@5	recall@10	MRR	latencia media (s)	coste medio (\$)
denso (base)	0.00	0.00	0.25	0.25	0.06	746.5 ms	0.00000
+ filtro metadatos	0.25	0.50	0.67	0.83	0.44	248.7 ms	0.00000

configuración	recall@1	recall@3	recall@5	recall@10	MRR	latencia media (s)	coste medio (\$)
+ BM25 (híbrido RRF)	0.25	0.50	0.58	0.67	0.40	251.1 ms	0.00000
+ reescritura de consulta	0.67	0.67	0.75	0.92	0.71	4898.8 ms	0.00192
todo	0.58	0.83	0.92	1.00	0.74	4903.1 ms	0.00192
+ reordenación (cross-encoder)	0.17	0.25	0.58	0.75	0.32	709.3 ms	0.00000
+ reescritura + reordenación	0.83	0.83	0.83	0.92	0.85	5201.8 ms	0.00192
todo + reordenación	0.83	0.83	0.83	0.92	0.85	5182.6 ms	0.00192
+ reescritura + reordenación (RRF)	0.67	0.75	0.92	0.92	0.74	5186.1 ms	0.00192
todo + reordenación (RRF)	0.67	0.92	0.92	0.92	0.78	5176.3 ms	0.00192

Y el contraste de cada fila contra la de partida:

Contraste contra «denso (base)» sobre recall@5, 12 preguntas con ancla. McNemar exacto, pareado.

configuración	recall@5	arregla	rompe	p
+ filtro metadatos	0.67	5	0	0.062 (n. s.)
+ BM25 (híbrido RRF)	0.58	4	0	0.125 (n. s.)
+ reescritura de consulta	0.75	6	0	0.031
todo	0.92	8	0	0.008
+ reordenación (cross-encoder)	0.58	4	0	0.125 (n. s.)
+ reescritura + reordenación	0.83	7	0	0.016
todo + reordenación	0.83	7	0	0.016
+ reescritura + reordenación (RRF)	0.92	8	0	0.008
todo + reordenación (RRF)	0.92	8	0	0.008

Lectura:

- **El filtro de metadatos es lo primero y casi lo más barato.** Filtrar *antes* de buscar, en vez de filtrar el orden que devuelve el índice, sube el recall sin coste ni latencia. Aun así no aguanta el contraste por sí solo: con doce preguntas con ancla, cinco aciertos nuevos y ningún fallo nuevo se quedan justo en el borde.
- **La reescritura de la consulta es la mejora grande, y cuesta dinero.** La pregunta llega en español y en lenguaje de persona; el corpus está en inglés y en lenguaje de abogado. Traducirla con el modelo antes de buscar es lo que parte la tabla en dos, y es la primera fila cuyo p-valor baja del 5 %. Añade una llamada al modelo y unos cinco segundos por consulta: está en la columna de coste y en la de latencia, no en una nota al pie.
- **El cross-encoder solo vale después de traducir.** Reordenar los candidatos con `ms-marco-MiniLM-L6-v2` sobre la consulta en español *empeora* —el modelo está entrenado en inglés—; sobre la consulta ya traducida sube el recall@1 y el MRR de forma clara. Es el mejor ejemplo de la práctica de que el orden de las mejoras importa tanto como las mejoras.

- **Reordenar puro sube arriba y puede hundir abajo.** El cross-encoder no recupera, solo reordena: cuando se equivoca, tira fuera del top-5 un pasaje que el recuperador ya tenía. Fusionar su orden con el del recuperador por RRF —el mismo RRF del híbrido— conserva el recall profundo y sigue ganando arriba. Para un agente que lee cinco fragmentos, la fila de RRF es la que hay que llevar.
- **BM25 no aportó.** El híbrido denso + léxico con RRF no mejora al denso con filtro en nuestro golden set. Es un arreglo razonable que no movió la métrica, y eso también es un resultado.

La tabla es **reproducible**: las reescrituras se cachean en disco con el modelo en la clave, porque el modelo no devuelve la misma reescritura ni con temperatura 0, y sin esa caché dos ejecuciones seguidas daban recalls distintos. Los aciertos de caché siguen sumando su coste y su latencia originales: si no, la segunda ejecución diría que reescribir es gratis.

5. Baseline contra sistema final

El baseline es el agente del día 10 — `miax_s2.baseline()` — con **el mismo modelo** que el sistema final: la tabla compara dos sistemas, no dos modelos. Se ejecutó y se **congeló** antes de tocar nada (`resultados/baseline/SELLO.json`, huellas SHA-256 de cada fichero); regenerarlo exige borrar el sello a mano y dejar constancia en el commit.

sistema	extractiva	numérica	comparativa	hueco	total	recall@5	coste medio	latencia media	tool calls/pregunta
baseline	83%	75%	0%	33%	55%	25%	1.234 ¢	17.7 s	3.4
final	83%	100%	83%	100%	90%	92%	1.652 ¢	22.3 s	2.6

sistema	aciertos	IC 95 % (Wilson)
baseline	11/20	[34%, 74%]
final	18/20	[70%, 97%]

McNemar exacto sobre las 20 preguntas comunes: baseline acierta y final falla en 0; al revés, 7. $p = 0.016$, la diferencia es significativa al 5 %.

La mejora aguanta el contraste. El sistema final no pierde ninguna pregunta que acertara el baseline, y la diferencia es significativa al 5 % con McNemar exacto. Los intervalos de Wilson de los dos sistemas apenas se tocan.

Qué cambió del baseline al final

- **Comparativas.** El baseline no acertaba ninguna. El prompt le dice ahora que haga dos consultas exactas, una por ejercicio, y que la respuesta lleve cifra de XBRL y cita del texto a la vez.
- **Huecos.** El baseline se inventó una cifra donde no había dato. El final no alucina ninguna: el prompt autoriza a decir «no está», y el guardarraíl no salta ante una respuesta de hueco.
- **Guardarraíl XBRL.** Salta cuando la cifra afirmada no cuadra con el hecho XBRL, le devuelve el desajuste al modelo y le deja corregir; nunca corrige él. La tabla del guardarraíl, en el detalle del final, dice cuántas veces saltó y cuántas de esas acabaron bien.
- **Citas.** La versión 2 del prompt dice cómo se cita: frase literal y su `chunk_id`. La mitad de los fallos de las extractivas eran citas parafraseadas o sin identificador.

- **Robustez.** Reintento ante errores transitorios del proveedor —una pregunta moría en cada corrida por un «Provider returned error»—, y si el modelo termina sin salida estructurada, se le pide en el mismo hilo.

¿Qué aporta el retrieval al agente?

La tabla de ablación de la sección 4 mide el buscador solo. Esta mide el agente entero con y sin las mejoras de búsqueda: la fila del medio es el sistema final con el retrieval del baseline —denso, sin filtro previo, sin reescritura y sin reordenar—.

sistema	extractiva	numérica	comparativa	hueco	total	recall@5	coste medio	latencia media	tool calls/pregunta
baseline	83%	75%	0%	33%	55%	25%	1.234 ¢	17.7 s	3.4
final-sin-mejoras-retrieval	67%	100%	83%	100%	85%	25%	1.805 ¢	22.3 s	2.8
final	83%	100%	83%	100%	90%	92%	1.652 ¢	22.3 s	2.6

Casi toda la mejora es del agente, no del buscador. El recall@5 pasa de un cuarto a más del noventa por ciento, y el acierto del agente sube una sola pregunta. Tiene explicación: el agente pasa sus propios filtros de emisor y ejercicio a `search_filings` y saca las cifras de XBRL, así que buena parte de lo que arregla el retrieval ya lo arreglaba él. Lo que el buscador sí aporta es eficiencia: con mejores fragmentos arriba, el agente hace menos llamadas y la pregunta sale algo más barata, aunque la reescritura añade la suya.

Es el resultado más útil de la práctica y el menos intuitivo: una mejora enorme en la métrica de un componente puede no llegar al sistema. Por eso se mide las dos cosas.

El baseline, en detalle

Resultados · baseline

- fecha: 2026-09-23 19:35 UTC
- modelo: openrouter:google/gemini-3.8-flash
- commit: 6800450
- preguntas: 20 (golden\golden_set.jsonl)
- acierto total: 11/20, IC 95 % [34%, 74%]

Resumen

sistema	extractiva	numérica	comparativa	hueco	total	recall@5	coste medio	latencia media	tool calls/pregunta
baseline	83%	75%	0%	33%	55%	25%	1.234 ¢	17.7 s	3.4

Por familia

familia	preguntas	aciertos	qué falló
numerica	8	6/8	fuelle_no_ninguna (2), cita_no_literal (1), alucinacion_sobre_hueco (1)
hueco	3	1/3	fuelle_no_ninguna (2), cita_no_literal (1), alucinacion_sobre_hueco (1)
comparativa	6	0/6	cifra_no_coincide (5), cita_no_literal (1), no_respalda_ancla (1)
extractiva	6	5/6	error de ejecución (1)

Trayectoria

trayectoria	preguntas	proporción
camino correcto	19	100%
camino equivocado, acierta	0	0%
camino equivocado, falla	0	0%
no aplica (sin herramienta_esperada)	1	—

Guardarraíl

guardarraíl	valor
preguntas con intervención del guardarraíl	0%
intervenciones totales	0
recuperadas tras intervenir (respuesta correcta)	0%
preguntas que agotaron el límite de llamadas	0
alucinaciones sobre hueco	1
abstenciones indebidas (dijo «no hay dato» y sí lo había)	0%

Retrieval

recall@1	recall@3	recall@5	recall@10	MRR
0%	0%	25%	25%	0.06

pregunta	puesto del ancla
g-r-005	> 10
g-r-006	> 10
g-r-007	> 10
g-r-008	> 10
g-r-009	> 10
g-r-010	> 10
g-r-011	5
g-r-012	> 10

pregunta	puesto del ancla
g-r-017	> 10
g-r-018	> 10
g-r-019	4
g-r-020	4

Por pregunta

id	familia	acierto	herramientas	qué falló
g-r-001	numerica	no	list_available > get_xbrl_fact > search_filings > search_filings > search_filings > get_xbrl_fact > get_xbrl_fact	cita_no_literal (1), alucinacion_sobre_hueco (1), fuente_no_ninguna (1)
g-r-002	numerica	sí	list_available > get_xbrl_fact > search_filings > get_xbrl_fact	
g-r-003	numerica	sí	get_xbrl_fact	
g-r-004	numerica	sí	list_available > get_xbrl_fact	
g-r-005	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > search_filings	cifra_no_coincide (1)
g-r-006	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > search_filings	cifra_no_coincide (1)
g-r-007	extractiva	sí	list_available > search_filings	
g-r-008	extractiva	sí	list_available > search_filings	
g-r-009	extractiva	error	—	error de ejecución (1)
g-r-010	extractiva	sí	list_available > search_filings	
g-r-011	extractiva	sí	list_available > search_filings > search_filings	
g-r-012	extractiva	sí	list_available > search_filings	
g-r-013	numerica	sí	list_available > get_xbrl_fact > search_filings > get_xbrl_fact > get_xbrl_fact	
g-r-014	numerica	sí	list_available > get_xbrl_fact	
g-r-015	numerica	sí	list_available > get_xbrl_fact	
g-r-016	numerica	no	list_available > get_xbrl_fact > search_filings > search_filings	fuente_no_ninguna (1)

id	familia	acierto	herramientas	qué falló
g-r-017	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > search_filings	cita_no_literal (1)
g-r-018	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > search_filings	no_respalda_ancla (1), cifra_no_coincide (1)
g-r-019	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > get_xbrl_fact > search_filings	cifra_no_coincide (1)
g-r-020	comparativa	no	list_available > get_xbrl_fact > get_xbrl_fact > search_filings > search_filings	cifra_no_coincide (1)

Tres cosas que se leen ahí y que orientan el trabajo de mejora:

- **Enruta bien y contesta mal.** La trayectoria es correcta en todas las preguntas en las que aplica: el agente va a XBRL cuando la pregunta es numérica y al texto cuando es un matiz. El problema no es la elección de herramienta.
- **Las comparativas son el agujero.** Ninguna sale. El patrón es siempre el mismo: consulta un ejercicio, consulta el otro y falla al combinarlos o arrastra la cifra de uno de los dos. Es lo que hay que atacar primero.
- **Hay una alucinación sobre hueco.** Con el dato ausente del corpus, el sistema dio una cifra igualmente. Es el fallo más caro de los que mide la tabla y es exactamente lo que el guardarraíl tiene que parar.

Una pregunta murió con un error del proveedor (`BadRequestResponseError`) y cuenta como fallo: la ejecución continúa y lo deja escrito en vez de interrumpirse, que es la regla del ejecutor. Está en la columna «qué falló» como error de ejecución.

6. Las diez preguntas ciegas

Se ejecutan en clase, contra el repositorio entregado y sin tocar código, con `evaluar("ciegas.jsonl")`. Su resultado y el delta contra el golden set propio se presentan en la sesión: `agente-10k informe` los escribe en `resultados/tablas/delta_ciegas.md`.

Si el acierto baja, la diferencia es la parte de la mejora que era memoria del conjunto con el que iteramos. Es un hallazgo, no un suspenso: lo que no se puede hacer es no medirlo.

7. Qué se probó y no funcionó

El enunciado pide esto explícitamente, y es la parte más barata de escribir y la más cara de callarse:

- **BM25 en el híbrido.** La fusión RRF de denso y léxico no mejora al denso con filtro de metadatos en nuestro golden set. La intuición era que los números y los nombres propios ("Item 7A", "60,922") le vendrían bien al léxico; el tokenizador los conserva y aun así no movió la métrica.
- **Reordenar sin traducir.** El cross-encoder sobre la consulta en español empeora el recall. No es que el modelo sea malo: es que está entrenado en inglés y se le estaba dando otra cosa.
- **Reordenar puro con el agente leyendo cinco fragmentos.** Sube el recall@1, que es la métrica bonita, y baja el recall@10, que es la que se lleva el agente cuando pide más contexto. Por eso la configuración recomendada es la fusionada y no la que gana la columna más vistosa.

8. Coste y latencia

Las dos columnas están en todas las tablas y no en una nota al pie, que es lo que pide el enunciado. Dos advertencias sobre cómo se calculan:

- **El coste se LEE de los metadatos del proveedor cuando viene.** OpenRouter no siempre lo manda; cuando falta, se estima con la tarifa de `miax_s2.PRECIOS_OPENROUTER` y la orden dice cuál de las dos cosas fue. No hay ninguna cifra de coste inventada.
- **La reescritura de la consulta se cobra aunque la sirva la caché.** La caché existe para que la tabla sea reproducible, no para que la técnica parezca gratis: cada acierto de caché vuelve a sumar el coste y la latencia de la llamada original.

9. Reproducir esto

```
pip install -e ".[dev]"
# descomprimir corpus_miax_2026.zip e indice_faiss.zip en data/corpus/
agente-10k diagnostico      # qué hay y qué falta
agente-10k validar-golden golden/golden_set.jsonl
agente-10k ablacion         # tabla de retrieval + contraste
agente-10k baseline        # ejecuta y CONGELA el baseline
agente-10k evaluar golden/golden_set.jsonl --etiqueta final
agente-10k informe         # las tablas, en resultados/tablas/
```

Ninguna clave de API está en el repositorio: se leen del entorno o de un `.env` que no se versiona, y `.env.example` dice cuáles hacen falta.